

Milestone 1

```
(base) C:\Users\nusra\Documents\nsc-course>python week3\profile_week3_m1.py

==== naive_profile.prof (top 10 cumulative) ====
Mon Mar  2 23:21:52 2026    naive_profile.prof

      262175 function calls in 4.948 seconds

Ordered by: cumulative time
List reduced from 19 to 10 due to restriction <10>

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
   1    0.000    0.000    4.948    4.948 {built-in method builtins.exec}
   1    0.000    0.000    4.948    4.948 <string>:1(<module>)
   1    0.711    0.711    4.948    4.948 C:\Users\nusra\Documents\nsc-course\week3\mandelbrot_week3.py:22(mandelbrot_naive)
262144    4.234    0.000    4.234    0.000 C:\Users\nusra\Documents\nsc-course\week3\mandelbrot_week3.py:10(mandelbrot_point)
   2    0.001    0.001    0.003    0.001 C:\Users\nusra\miniforge3\Lib\site-packages\numpy\_core\function_base.py:26(linspace)
   2    0.001    0.000    0.001    0.000 {built-in method numpy.arange}
   2    0.000    0.000    0.000    0.000 {built-in method _operator.index}
   2    0.000    0.000    0.000    0.000 {method 'astype' of 'numpy.ndarray' objects}
   1    0.000    0.000    0.000    0.000 {built-in method numpy.zeros}
   1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}

==== numpy_profile.prof (top 10 cumulative) ====
Mon Mar  2 23:21:52 2026    numpy_profile.prof

       74 function calls in 0.630 seconds

Ordered by: cumulative time
List reduced from 39 to 10 due to restriction <10>

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
   1    0.000    0.000    0.630    0.630 {built-in method builtins.exec}
   1    0.001    0.001    0.630    0.630 <string>:1(<module>)
   1    0.588    0.588    0.629    0.629 C:\Users\nusra\Documents\nsc-course\week3\mandelbrot_week3.py:39(mandelbrot_numpy)
   1    0.000    0.000    0.038    0.038 C:\Users\nusra\miniforge3\Lib\site-packages\numpy\lib\_function_base_impl.py:5134(meshgrid)
   1    0.000    0.000    0.036    0.036 C:\Users\nusra\miniforge3\Lib\site-packages\numpy\lib\_stride_tricks_impl.py:481(broadcast_arrays)
   1    0.035    0.035    0.035    0.035 C:\Users\nusra\miniforge3\Lib\site-packages\numpy\lib\_stride_tricks_impl.py:413(_broadcast_shape)
   1    0.003    0.003    0.003    0.003 C:\Users\nusra\miniforge3\Lib\site-packages\numpy\_core\numeric.py:98(zeros_like)
   3    0.000    0.000    0.002    0.001 C:\Users\nusra\miniforge3\Lib\site-packages\numpy\lib\_function_base_impl.py:5280(<genexpr>)
   2    0.002    0.001    0.002    0.001 {method 'copy' of 'numpy.ndarray' objects}
   2    0.001    0.000    0.001    0.000 C:\Users\nusra\miniforge3\Lib\site-packages\numpy\lib\_stride_tricks_impl.py:340(_broadcast_to)
```

1. Which function takes most total time?

In the naive version, `mandelbrot_point` takes the most time: **4.310 s** cumulative, out of **5.034 s** total.

2. Are there functions called surprisingly many times?

Yes. `mandelbrot_point` is called **262,144 times**, which matches one call per pixel for a **512×512** grid. That huge call count explains why naive Python is slow.

3. How does NumPy profile compare to naive?

NumPy has **far fewer Python-level calls** and much lower runtime:

- Naive: **262,175 function calls, 5.034 s runtime**
- NumPy: **74 function calls, 0.619 s runtime**

This shows NumPy pushes most work into compiled routines instead of Python loops.

4. Where does NumPy spend its time?

Most time is inside `mandelbrot_numpy` function: **0.618 s cumulative** (almost all the 0.619 s total). Within NumPy internals, the next biggest costs shown are `meshgrid` (~0.038 s) and broadcasting-related functions (`broadcast_arrays` ~0.036 s, `_broadcast_shape` ~0.035 s).

Milestone 2

```
(base) C:\Users\nusra\Documents\nsc-course>kernprof -l -v week3\run_m2.py
(512, 512) int32
Wrote profile results to 'run_m2.py.lprof'
Timer unit: 1e-06 s

Total time: 9.64811 s
File: C:\Users\nusra\Documents\nsc-course\week3\mandelbrot_week3_m2.py
Function: mandelbrot_naive at line 21

Line #      Hits          Time    Per Hit   % Time  Line Contents
=====
21          1             0.0         0.0     0.0      @profile # <-- Milestone 2: line profiler decorator (works only with kernprof)
22          1             0.0         0.0     0.0      def mandelbrot_naive(xmin, xmax, ymin, ymax, width, height, max_iter=100):
23          1             0.0         0.0     0.0          """
24          1             0.0         0.0     0.0          Naive Mandelbrot: Python loops over all pixels; calls mandelbrot_point per pixel.
25          1             0.0         0.0     0.0          """
26          1             0.0         0.0     0.0          x = np.linspace(xmin, xmax, width)
27          1             0.0         0.0     0.0          y = np.linspace(ymin, ymax, height)
28          1             0.0         0.0     0.0
29          1             0.0         0.0     0.0          result = np.zeros((height, width), dtype=np.int32)
30          1             0.0         0.0     0.0
31          513           169.5         0.3     0.0          for i in range(height):
32          262656       80603.0         0.3     0.8              for j in range(width):
33          262144       735846.0         2.8     7.6                  c = x[j] + 1j * y[i]
34          262144       8830606.9        33.7    91.5                      result[i, j] = mandelbrot_point(c, max_iter)
35          1             0.0         0.0     0.0
36          1             0.0         0.0     0.0          return result
```

1. cProfile on naive vs NumPy: How many functions appear in each profile? What does this difference tell you about where the work actually happens?

- Naive: 262,175 function calls, 5.034 s total runtime
- NumPy: 74 function calls, 0.619 s total runtime

The naive implementation performs a very large number of Python-level function calls, especially repeated calls to `mandelbrot_point`. In contrast, the NumPy implementation makes very few Python-level calls because most computation happens inside optimized compiled NumPy routines, not in Python loops.

2. line profiler on naive: Which lines dominate runtime? What fraction of total time is spent in the inner loop?

The dominant lines in `mandelbrot_naive` are:

- `result[i, j] = mandelbrot_point(c, max_iter)` → 91.9%
- `c = x[j] + 1j * y[i]` → 7.3%

Together, the inner loop accounts for approximately **99.2% of total runtime**, showing that nearly all execution time is spent in the per-pixel computation.

3. Based on your profiling results: why is NumPy faster than naive Python?

NumPy is faster because it avoids explicit Python loops and repeated Python function calls. Instead, operations are **vectorized and executed** inside optimized compiled code. This drastically reduces interpreter overhead and lowers the number of Python-level function calls (74 vs 262,175).

4. What would you need to change to make the naive version faster?

The line profiler shows that almost all runtime is spent inside the inner loop and the repeated call to `mandelbrot_point`. To make the naive version faster, the Python-level inner loop must be optimized or moved into compiled code. This can be achieved using **JIT compilation (Numba) or vectorized operations (NumPy)** to eliminate Python interpreter overhead in the pixel-wise iteration.

Milestone 3

```
(base) C:\Users\nusra\Documents\nsc-course>python week3\benchmark_m3.py
Naive: 4.826 s
NumPy: 0.708 s
Numba (serial): 0.024 s
```

1. What are the runtimes of naive, NumPy, and Numba implementations?

- Naive Python: 4.785 s
- NumPy: 0.575 s
- Numba (serial @njit): 0.019 s

2. What is the speedup of Numba compared to naive Python?

- NumPy speedup: $\approx 8.32\times$
- Numba speedup: $\approx 251.8\times$

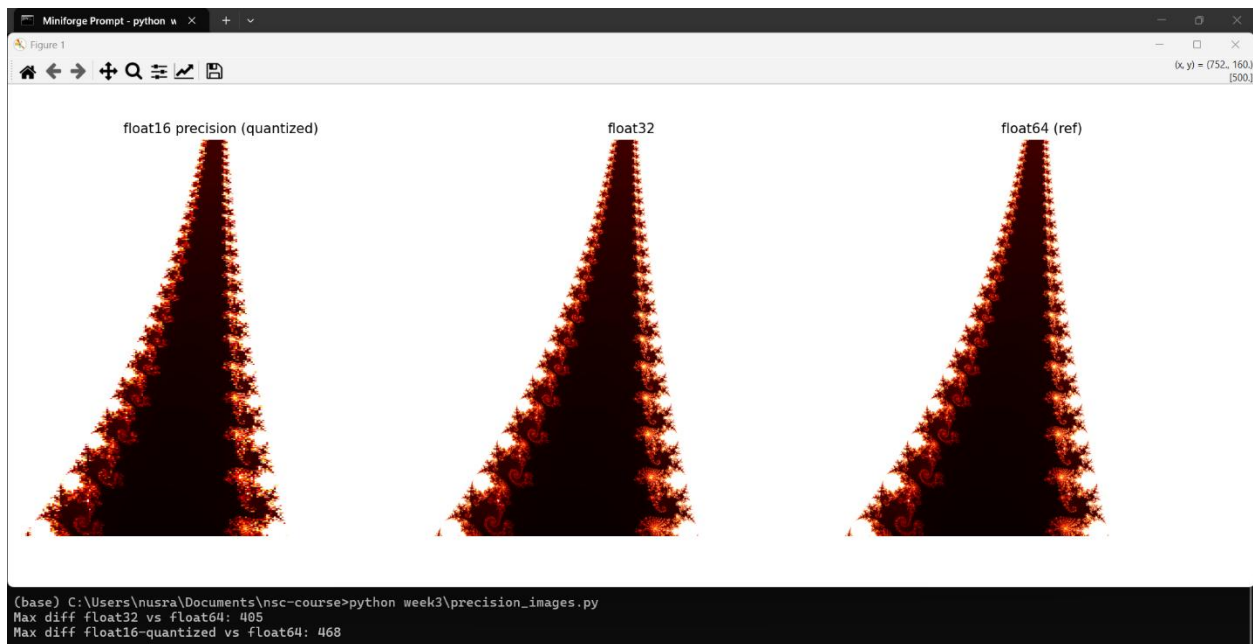
3. Why is Numba faster than naive Python?

Numba compiles the nested loops and the inner while loop into optimized machine code, eliminating Python interpreter overhead inside the pixel-wise computation.

4. Why does Mandelbrot benefit especially from Numba?

The Mandelbrot algorithm contains an **early-exit loop** (points escape at different iteration counts). Numba efficiently compiles this loop structure, allowing fast execution with early termination.

Milestone 4



1. Speed: Does float32 actually run faster than float64 on your hardware? By how much?

No — they are essentially the same on my CPU.

- float32: **0.074575 s**
- float64: **0.074407 s**

Difference: **0.000168 s** ($\sim 0.23\%$) → float64 was slightly faster, but the difference is within noise.

2. float16: Try it with NumPy — is it faster than float32?

- In **NumPy**, float16 is often **not faster** because many operations are promoted to **float32** internally.
- In **Numba**, native float16 may **fail to compile** for this kernel on CPU. On my setup, I used **float16 quantization** (inputs rounded to float16 but computed as float32).

Runtime was:

- **float16** (quantized, computed as float32): **0.074424 s**
This is effectively the same as **float32**, so **float16** did **not** provide a speed advantage.

3. Visual quality: Can you see artefacts with float16? What about float32?

In the selected zoom region, **float32 and float64 look the same**.

Float16 (quantized) also looks **very similar at this zoom**, but artefacts are more likely to appear at **deeper zoom / higher max_iter** due to reduced precision.

4. Recommendation: Which precision would you choose for production use, and why?

For CPU production use, I would choose **float32**: it gives good precision and shows **no meaningful performance disadvantage** compared to float64 in my tests. I would avoid float16 on CPU because it provides **no speed benefit here** and reduces numerical precision.

Milestone 5

```
(base) C:\Users\nusra\Documents\nsc-course>python week3\benchmark_parallel.py
Serial Numba: 0.814 s
Parallel Numba: 0.292 s
Speedup: 2.79x
```

1. Experiment 1: @jit vs @njit — what is the performance difference?

- @jit: **0.297548 s** (avg over 5 runs)
- @njit: **0.297865 s** (avg over 5 runs)

Speed ratio (jit/njit): **1.00×** (no meaningful difference).

Reason: @jit compiled in **nopython mode** on my setup (jit **nopython signatures exists**), so it behaved like @njit.

2. Experiment 2: Parallel Numba — what is the observed speedup?

- Serial Numba: **0.832 s**
- Parallel Numba: **0.304 s**

Speedup: **2.73×**

3. **Experiment 3: Scaling with grid size — does speedup scale with problem size?**

- **512²**: serial **0.051 s**, parallel **0.019 s**, speedup **2.63×**
- **1024²**: serial **0.203 s**, parallel **0.071 s**, speedup **2.85×**
- **2048²**: serial **0.812 s**, parallel **0.282 s**, speedup **2.88×**

Speedup increases slightly with larger grids but saturates below ideal linear scaling.

4. **Memory warning — what happens when grid size increases?**

Memory scales as $N^2 \times d$ bytes per array (**d=8 for float64, d=4 for float32**).

For example (float64 per array):

- **1024² $\approx$ 8 MB, 4096² $\approx$ 128 MB, 8192² $\approx$ 512 MB.**

NumPy may hold multiple arrays at once (C, Z, masks, result), so total memory can be **3–5×** higher, making very large grids memory-heavy.